

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 1/22
--	--	-----------------------------

EDF - R&D

DEPARTEMENT "MANAGEMENT DES
RISQUES INDUSTRIELS"



Avril 2019

GROUPE : ANALYSE DES RISQUES
DES ORGANISATIONS ET SYSTEMES

M. BOUISSOU

**Manuel de développement de bases de connaissances
pour le logiciel KB3 v3**

HT-52/03/042/J

Documents associés :

Document décrivant la syntaxe du langage FIGARO

Manuel de référence du langage FIGARO ; Tutoriel d'utilisation de l'interface VisualFigaro

Résumé :

L'outil KB3 a pour but d'automatiser les études de sûreté de fonctionnement, grâce à l'utilisation de bases de connaissances. Il permet des gains très importants dans la productivité des études de systèmes complexes et permet à des non spécialistes de la sûreté de fonctionnement de les réaliser. Les bases de connaissances sont constituées de deux parties, l'une relative à la sémantique des modèles graphiques saisis par l'utilisateur final, décrite en langage FIGARO, et l'autre destinée à paramétrer l'interface graphique. Nous donnons dans ce manuel une procédure permettant de développer une nouvelle base de connaissances, en "couches" successives, partant du cœur en langage FIGARO et amenant progressivement à une application finalisée et conviviale.

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 2/22
--	--	-----------------------------

SYNTHESE

L'outil KB3 a pour but d'automatiser les études de sûreté de fonctionnement, grâce à l'utilisation de bases de connaissances. Il permet des gains très importants dans la productivité des études de systèmes complexes et permet à des non spécialistes de la sûreté de fonctionnement de les réaliser. Les bases de connaissances sont constituées de deux parties, l'une relative à la sémantique des modèles graphiques saisis par l'utilisateur final, décrite en langage FIGARO, et l'autre destinée à paramétrer l'interface graphique. Nous donnons dans ce manuel une procédure permettant de développer une nouvelle base de connaissances, en "couches" successives, partant du cœur en langage FIGARO et amenant progressivement à une application finalisée et conviviale.

Afin de réduire les coûts de développement des outils autour de KB3, la procédure que nous donnons repose essentiellement sur l'utilisation d'outils standard du marché, complétés par le programme spécifique VisualFigaro qui permet une aide à la saisie et des contrôles sophistiqués de cohérence des informations saisies.

Les étapes du développement sont en résumé :

- La construction du "noyau dur" de la bdc
 - La construction en langage FIGARO d'une première ébauche de la bdc
 - Les premiers tests de comportement des modèles construits grâce à une interface réduite au minimum
 - Un premier paramétrage de l'interface graphique, destiné à faciliter la saisie de modèles par un remplissage automatique des interfaces des objets FIGARO à partir des liens tracés par l'utilisateur
 - L'écriture des algorithmes permettant de construire des "objets déduits" à partir des objets saisis via l'IHM
- Les améliorations de la facilité d'utilisation
 - Choisir ou créer des icônes et styles de liens
 - Exploiter l'héritage dans le fichier de définition de l'IHM
 - Ajouter des modes de visualisation
- Le "blindage" de l'application
 - Comment protéger l'application contre les mauvaises manipulations par l'utilisateur
- La rédaction d'aides pour l'utilisateur
- La maintenance de la bdc en assurant la compatibilité avec les études déjà saisies

SOMMAIRE

1	INTRODUCTION.....	4
2	ELEMENTS CONSTITUANTS D'UNE BDC	4
3	L'ENVIRONNEMENT DEVELOPPEUR POUR KB3.....	5
3.1	LES LOGICIELS	5
3.2	CONFIGURATION DE JEDIT (IMPORTANT !)	5
3.3	LES ICONES STANDARD	6
4	ETAPES DE LA CONSTRUCTION D'UNE NOUVELLE BDC.....	7
4.1	CREATION DES FICHIERS DE DEPART	7
4.2	CONSTRUCTION DU CŒUR DU MODELE (EN FIGARO 1).....	7
4.3	PREMIERS TESTS EN MODE TEXTE PUR	8
4.4	PREMIERS TESTS AVEC L'INTERFACE GRAPHIQUE DE KB3.....	9
4.4.1	<i>Génération automatique d'une première version du fichier .bdc</i>	9
4.4.2	<i>Spécification des traitements associés à la bdc</i>	10
4.4.3	<i>Premiers tests</i>	12
4.5	SPECIFIER LES POINTS DE CONNEXION	12
4.6	RENSEIGNER LES CONTRAINTES DE CONNEXION ET LES REGLES DE REMPLISSAGE DES INTERFACES	13
4.7	SPECIFIER LES ALGORITHMES D'OBJETS DEDUITS	15
4.8	CHOISIR OU CREER DES ICONES ET STYLES DE LIENS	16
4.8.1	<i> Icônes</i>	16
4.8.2	<i> Liens</i>	16
4.9	EXPLOITER L'HERITAGE DANS LE FICHIER .BDC.....	16
4.10	AJOUTER DES MODES DE VISUALISATION ET VARIANTES GRAPHIQUES.....	17
4.11	AJOUTER DES CALQUES.....	17
4.12	VERROUILLER L'APPLICATION.....	18
4.13	REDIGER LES AIDES A L'UTILISATEUR	18
5	MODIFICATION D'UNE BDC EXISTANTE.....	18
5.1	PRINCIPES GENERAUX	18
5.2	COMMENT RECHARGER LES ETUDES SAISIES AVEC UNE VERSION PRECEDENTE DE BDC ?.....	19
6	CONCLUSION.....	20
7	ANNEXE A : PASSAGE DE PARAMETRES LORS DU LANCEMENT D'UN CODE EXTERNE PAR KB3.....	21

Répertoire des modifications du document

Référence	Désignation des modifications	Observations
A	Version initiale diffusée officiellement	
B, C, D, E...	Versions mises à jour pour tenir compte des évolutions de KB3 et des outils d'édition de bases de connaissances. Pas de diffusion officielle.	

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 4/22
----------------------------	--	-----------------------------

1 Introduction

L'outil KB3 a pour but d'automatiser les études de sûreté de fonctionnement, grâce à l'utilisation de bases de connaissances (bdc). Il permet des gains très importants dans la productivité des études de systèmes complexes et permet à des non spécialistes de la sûreté de fonctionnement de les réaliser. Les bases de connaissances sont constituées essentiellement de deux parties, l'une relative à la sémantique des modèles graphiques saisis par l'utilisateur final, décrite en langage FIGARO, et l'autre destinée à paramétrer l'interface graphique. Nous donnons dans ce manuel une procédure permettant de développer une nouvelle base de connaissances, en "couches" successives, partant du cœur en langage FIGARO et amenant progressivement à une application finalisée et conviviale.

Ce manuel s'adresse à des développeurs déjà au fait des concepts que l'on trouve dans une base de connaissances pour KB3. Il est à prendre comme une sorte de check-list pratique destinée à faire gagner du temps. Il est donc volontairement succinct.

Dans une première partie, nous décrirons les éléments constituant d'une base de connaissances, et l'ensemble des outils que nous désignons sous le nom "d'environnement développeur pour KB3".

Ensuite nous donnerons une procédure qui, suivie pas à pas, permet la construction progressive d'une base de connaissances en exploitant ces outils. On peut faire une analogie entre cette procédure et la notion de "couches" de finition, permettant de passer d'une maquette réalisée en quelques heures à une application parfaitement aboutie et très conviviale, munie d'une aide en ligne simple et d'une documentation.

Nous montrerons enfin que la modification d'une bdc existante peut également s'appuyer sur cette procédure.

Remarque : il existe deux documents servant de référence sur le langage FIGARO. Le manuel de référence est un document classique, qui peut être lu de manière séquentielle ; il contient de nombreux exemples. Le document « Syntaxe du langage Figaro » est une définition exhaustive de la syntaxe par des diagrammes syntaxiques, aussi concise que possible afin de permettre une navigation rapide.

2 Éléments constitutants d'une bdc

Une base de connaissances se compose des fichiers suivants :

- Base.fi : description des types en langage Figaro
- Base.bdc : paramétrage de l'interface graphique, des algorithmes calculant les objets déduits, et des menus propres à la bdc. Ce fichier est en syntaxe XML, et doit respecter un schéma bien défini. Ce schéma est décrit dans le fichier Bdcfr.xsd ou Bdceng.xsd suivant qu'on va travailler en Français ou en Anglais
- Pour chaque type de la bdc ayant une représentation graphique :
 - Icône_type_i.ico : icône en bitmap (format 24x24 **impératif**) destinée à apparaître dans la palette située dans la sous-fenêtre de gauche de l'interface de KB3 ; c'est pourquoi il doit y en avoir une et une seule par type Figaro que l'on souhaite faire apparaître dans cette palette,
 - Icône_type_i.sym : icône dans un format propriétaire à la bibliothèque graphique Stingray qui a servi de base au développement de KB3 v3. Eventuellement, plusieurs icônes .sym peuvent être associées à un type Figaro : l'une ou l'autre de ces variantes graphiques sera affichée sur un schéma de système en fonction de conditions exprimées en langage Figaro.
- Un ou plusieurs fichier(s) d'aide (.doc ou .pdf).

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 5/22
--	--	-----------------------------

3 L'environnement développeur pour KB3

3.1 Les logiciels

La procédure que nous allons donner dans cette note s'appuie sur une collection d'outils gratuits qui ont été choisis pour leur efficacité dans différentes fonctions qui sont nécessaires à la construction de la bdc. Ces outils existent tous en version "portable", donc on peut les installer sans droit d'administration (sauf pour Java).

L'environnement "minimal" se compose de (dans l'ordre des installations à faire) :

- L'environnement d'exécution de Java,
- L'éditeur de texte en open source Jedit et des compléments à Jedit regroupés dans un installateur appelé "VisualFigaro".

Logiciels complémentaires (faciles à trouver sur Internet, sauf Designer qui est livré systématiquement avec KB3) permettant de travailler sur les bases de connaissances KB3 :

- Designer pour éditer les icônes en .sym. Designer est le **seul** outil permettant d'éditer les icônes dans ce format, propre à la bibliothèque graphique Stingray. Il est systématiquement livré avec KB3 (c'est l'exécutable Designer.exe présent dans le répertoire d'installation de KB3).
- IcoFX, AWicons lite etc. pour éditer les icônes en .ico. Ces outils offrent un mode de visualisation simultané de l'icône en gros (pour l'édition) et en réduction (pour que l'on voie en temps réel l'effet produit) ; ils permettent l'utilisation d'une palette réduite à 16 couleurs, le dessin de formes géométriques (cercles...) et de texte.
- QxmlEdit est un éditeur simple pour les fichiers XML. Il a l'avantage d'être léger à installer et riche en fonctions. Il présente clairement la structure arborescente du fichier XML et sépare bien les "tags" du contenu des champs. Il vérifie la conformité du fichier au schéma associé, ce qui peut être utile pour chercher les erreurs dans le fichier .bdc. Il permet aussi de visualiser graphiquement sous forme d'arbre les schémas tels que bdcfr.xsd et bdceng.xsd.

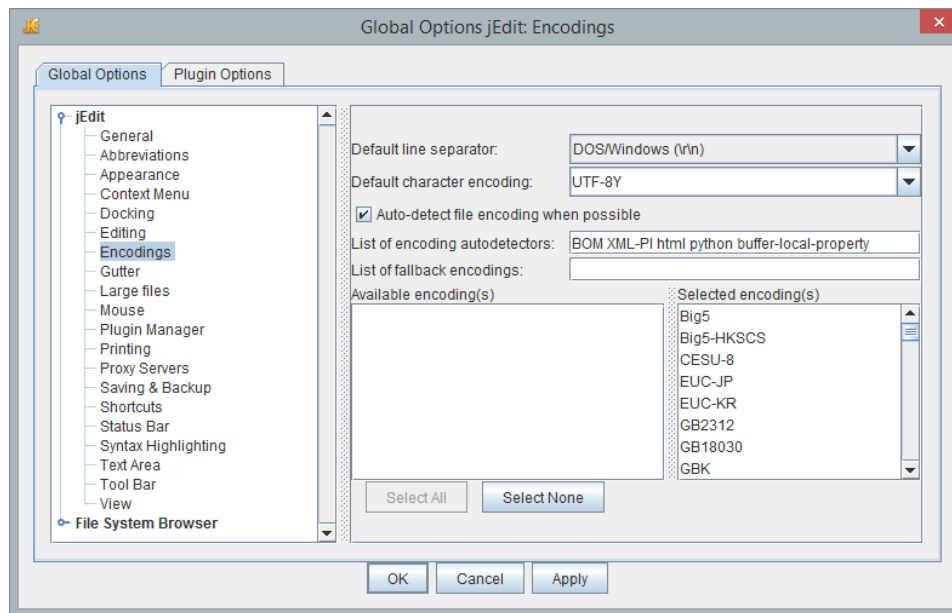
Le premier travail à effectuer est évidemment l'installation de l'environnement développeur. L'installation de tous les outils cités ci-dessus est très rapide. Le principal installateur (Jedit + VisualFigaro) qui contient une version « portable », c'est-à-dire entièrement autonome et ne nécessitant pas les droits d'administration est disponible sur Sourceforge, à l'adresse : <http://sourceforge.net/projects/visualfigaro/>.

3.2 Configuration de Jedit (important !)

Il est primordial de bien choisir quelques options de Jedit, pour une utilisation confortable de VisualFigaro.

Pour éviter que les lettres accentuées soient transformées en caractères illisibles, il faut vérifier que dans « Utilities / Global options / Jedit / Encodings » (cf. copie d'écran ci-dessous) l'option d'encodage par défaut est UTF8-Y ou UTF-8 (en principe c'est fait par l'installation, mais il peut y avoir une interférence créée par un paramétrage de Jedit déjà installé sur la machine).

Par ailleurs, dans la rubrique « Gutter » de ce même panneau d'options, il faut cocher la case « line numbers » car l'affichage des numéros de ligne sera utile pour trouver rapidement les erreurs de syntaxe signalées par le vérificateur de base de connaissances.



Dans la rubrique « Editing », il faut prendre l'option « indent » pour « folding mode », et prendre l'option « Initially collapse folds with level 1 » qui permet d'avoir une vue initiale d'une base de connaissances masquant les détails. Il suffit de faire un clic sur le triangle au regard d'un ensemble de lignes masquées pour faire apparaître un niveau de détail supplémentaire (shift + clic si on veut « tout déplier » en une seule fois). Cette fonction de Jedit est particulièrement pratique pour éditer le fichier .bdc (à condition de bien maintenir ses indentations).

On peut choisir la langue des menus de jEdit dans la rubrique « Utilities / Global options / jEdit / General ».

3.3 Les icônes standard

Pour éviter des duplications à l'infini des icônes, cette bibliothèque d'icônes "de base" est fournie au développeur KB3. Leurs noms doivent respecter les règles suivantes :

Ces icônes portent des noms en anglais si elles sont bilingues ; si elles ne le sont pas, la version française porte le même nom que la version anglaise, avec un suffixe _f, de façon qu'elles se suivent par ordre alphabétique.

Exemple : and_gate / and_gate_f

Le nom d'une icône doit être le plus descriptif possible, sans faire référence à un contexte particulier : on dira "white_circle" plutôt que "state" pour désigner une icône pouvant représenter un état dans un graphe de Markov. Il ne doit pas comporter de blanc.

Tant qu'une icône n'existe qu'en une couleur, inutile de préciser cette couleur dans le nom. Idem pour la taille.

On aura des variantes par couleurs simples : white, black, yellow, blue, red, green, etc...

On aura au maximum trois tailles : small, med(ium), big.

La taille moyenne correspondra à une image inscrite dans un carré de 48x48 pixels (2 fois la taille des .ico). Sera considéré comme "small" toute taille inférieure à cette limite, et comme "big" toute taille supérieure. Ce qualificatif de taille ne s'applique qu'à l'icône vectorielle : le même fichier .ico peut être associé à trois icônes vectorielles différentes.

Un nom donnera, dans cet ordre (et séparés par des "_") :

- la taille (obligatoire),
- la couleur (facultatif),
- la forme géométrique (obligatoire),
- le suffixe éventuel _f pour la langue.

Exemples : med_green_square, big_red_circle, med_and_gate, med_and_gate_f

Cas des icônes de liens : pour les liens, il n'y a pas de fichier .sym.

Le nom d'un fichier .ico de lien comprendra obligatoirement les éléments suivants, séparés par des "_" :

- type de ligne (solid, dash, dotted) (dans le fichier .bdc : TRAIT_PLEIN, POINTILLE, TIRET)
- couleur (mêmes conventions que pour les icônes de nœuds)
- type d'extrémités, sous la forme d'un abrégé, pris dans le tableau suivant (les deux dernières lignes correspondent à des lignes comportant une flèche en leur milieu) :

LIGNE_LIGNE	ll
LIGNE_TROIS_POINTS	l3p
LIGNE_QUATRE_POINTS	l4p
TROIS_POINTS_LIGNE	3pl
TROIS_POINTS_TROIS_POINTS	3p3p
TROIS_POINTS_QUATRE_POINTS	3p4p
QUATRE_POINTS_LIGNE	4pl
QUATRE_POINTS_TROIS_POINTS	3p4p
QUATRE_POINTS_QUATRE_POINTS	4p4p
LIGNE_TROIS_POINTS_LIGNE	l3pl
LIGNE_QUATRE_POINTS_LIGNE	l4pl

- épaisseur (entier : 1, 2,...)

Exemples : solid_green_ll_2, dash_yellow_3p3p_1...

4 Etapes de la construction d'une nouvelle bdc

Préambule : le présent document complète (sur les questions "de fond") les informations limitées à l'aspect pratique de manipulation de l'IHM décrites (avec de nombreuses copies d'écran) dans le document intitulé "Tutoriel de l'utilisation de VisualFigaro".

4.1 Création des fichiers de départ

Choisir un répertoire de développement et y créer les fichiers nécessaires à partir de VisualFigaro, par la commande : "KB management/New KB".

Cela crée un répertoire de nom "nom_bdc", contenant :

- un répertoire "icons", contenant un ensemble minimal d'icônes "standard" (notamment des formes géométriques simples),
- le fichier de schéma Bdcfr.xsd ou Bdceng.xsd suivant qu'on a choisi la langue Français ou Anglais,
- un fichier "nom_bdc.fi" initialisé par quelques lignes paramétrables dans le fichier "copiedFiles.xml" de l'environnement VisualFigaro.

4.2 Construction du cœur du modèle (en FIGARO 1)

Il s'agit de la base de connaissances en langage FIGARO d'ordre 1, à écrire en utilisant éventuellement l'aide syntaxique disponible à partir de l'arborescence de gauche par un clic droit.

Il est important de vérifier souvent la validité de la bdc en cliquant sur le grand bouton vertical.

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 8/22
--	--	-----------------------------

Supposons que nous ayons saisi la base de connaissances suivante :

```
( * Base de connaissances en langage FIGARO  * )
( *
    Cette bdc sert à modéliser des réseaux de télécoms
*)
TYPE GLOBAL_;
CONSTANTE
    lambda DOMAINE REEL ROLE CONCEPTION PAR_DEFAULT 1.E-8;
( * ----- *)
OBJET_SYSTEME Parametres EST_UN GLOBAL_;
( * ----- *)
TYPE composant;
EFFET relie;
PANNE DF;
OCCURRENCE
IL_PEUT_SE_PRODUIRE
DEFAILLANCE DF LOI EXP(lambda(Parametres));
( * ----- *)
TYPE noeud SORTE_DE composant ;
CONSTANTE
    source DOMAINE BOOLEEN PAR_DEFAULT FAUX;
INTERACTION
SI source ET MARCHE
ALORS relie ;
( * ----- *)
TYPE arete_uni_dir SORTE_DE composant ;
INTERFACE
    depart GENRE noeud CARDINAL 1;
    arrivee GENRE noeud CARDINAL 1;
INTERACTION
SI relie DE depart ET MARCHE
ALORS relie DE arrivee ;
( * ***** FIN ***** *)
```

4.3 Premiers tests en mode texte pur

Pour tester la base de connaissances en cours de construction, il faut pouvoir définir rapidement des petits « systèmes » c'est-à-dire des ensembles d'objets dont on précise les interfaces, les valeurs de caractéristiques constantes (telles que les taux de défaillances) etc.

Il est possible de le faire (depuis la v2.00 de Visual Figaro) en restant en mode purement textuel, donc sans perdre de temps à paramétrer l'interface graphique de KB3. Cette possibilité est intéressante car il arrive souvent qu'on remette en question certains choix faits dans la définition des classes, et dans ce cas la mise à jour des informations relatives aux aspects graphiques peut faire perdre du temps.

A l'aide du menu "Object Management" de Visual Figaro, on peut créer un modèle, c'est-à-dire un fichier contenant des descriptions d'objets. La commande "New Object" lance un petit wizard qui permet de créer facilement un objet à partir du choix de la classe à instancier et de quelques informations saisies dans un dialogue. A la fin du dialogue, l'objet est écrit dans le fichier modèle, en syntaxe Figaro. On peut évidemment aussi écrire directement dans le fichier !

Une fois que le fichier contenant le modèle est complet, on peut lancer diverses commandes dessus :

- La vérification (Check Model);
- L'instanciation en FIGARO 0 (Generate Figaro 0);
- Le lancement du débogueur interactif (Run Fig0debug);
- La génération d'un arbre de défaillances (Generate Fault Tree);

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 9/22
--	--	-----------------------------

- Le lancement des outils de calcul Figseq et YAMS¹.

Chacun de ces traitements (à part la vérification) commence par présenter une fenêtre de choix d'options. Les choix à faire sont faciles à comprendre, à l'exception peut-être du choix des groupes de règles à instancier. Il faut en particulier penser à sélectionner le groupe SANS_NOM si cela est pertinent. Si aucun groupe n'est sélectionné dans la liste, les règles de *tous* les groupes, y compris le groupe SANS_NOM sont instanciées.

Les options proposées ne comprennent pas toutes les possibilités qui existent à partir de l'IHM de KB3, mais seulement les plus courantes, suffisantes au début de la mise au point de la base de connaissances.

4.4 Premiers tests avec l'interface graphique de KB3

Le travail en mode purement textuel s'avère laborieux s'il faut définir de nombreux objets avec leurs interfaces. Il devient alors difficile d'avoir une vue globale et synthétique du modèle que l'on manipule. Si c'est le cas, il est temps de passer à l'utilisation de KB3 pour saisir les modèles de manière graphique.

Pour cela, il faut charger la base de connaissances dans KB3, ce qui n'est possible qu'avec un fichier .bdc contenant un minimum d'informations.

4.4.1 Génération automatique d'une première version du fichier .bdc

Par le menu « KB Management / Class Hierarchy » du plugin VisualFigaro, générer une représentation de la hiérarchie des classes sous forme de texte. Dans notre exemple, on obtient ceci :

```
-FIGARO
  |-GLOBAL_
  |-composant
  |   |-noeud
  |   |-arete_uni_dir
```

S'en servir pour aller ajouter les mots-clés GR_LIEN ou GR_NOEUD juste après les noms des types (au plus haut niveau possible dans la hiérarchie) pour différencier les liens des nœuds.

Dans notre exemple, cela nous amène à modifier deux lignes de la base de connaissances :

```
TYPE noeud GR_NOEUD SORTE_DE composant ;
TYPE arete_uni_dir GR_LIEN SORTE_DE composant ;
```

Générer le .bdc à partir du .fi par la commande : "Edit XML File" du plugin, en prenant l'option de génération de données par défaut (l'option "Start from scratch" est déconseillée aux non experts, car elle nécessite de bien connaître les données minimales à saisir pour pouvoir charger la base de connaissances dans KB3). Cette opération est à faire une seule fois pour une base de connaissances donnée (d'ailleurs, elle n'est proposée par VisualFigaro que si le fichier .bdc est absent quand on lance la commande d'édition de ce fichier). Par la suite, si l'on ajoute des types dans le fichier .fi, le plus rapide est habituellement de faire du copier-coller à partir de types déjà déclarés dans le fichier .bdc à l'aide de QxmlEdit (ou jEdit) pour les y ajouter aussi.

De toute façon, à partir de ce point, il est conseillé d'ouvrir périodiquement le fichier .bdc dans QxmlEdit. Ainsi on a les éventuels messages d'erreur dus à un non-respect du schéma avant de tenter de charger la base de connaissances dans KB3. Ces messages sont dus pour la plupart à des structures renseignées de manière incomplète.

Le .bdc généré ne contient pas les types intermédiaires (qui ne sont pas des feuilles dans la hiérarchie des types FIGARO) : il sera possible d'en ajouter éventuellement pour factoriser des informations (cf. § 4.9) communes à plusieurs types terminaux.

Chaque type terminal **noeud** est muni de "ports" (points désignés par leurs coordonnées en X et Y dans le rectangle circonscrit à l'icône du nœud) standard "in" et "out" auxquels tous les liens ont l'autorisation de s'accrocher.

¹ Pour Figseq et YAMS, il faut installer ces codes séparément de Visual Figaro, et se référer aux indications du §4.4.2 pour éventuellement renseigner à la main le fichier KB3Configuration.xml.

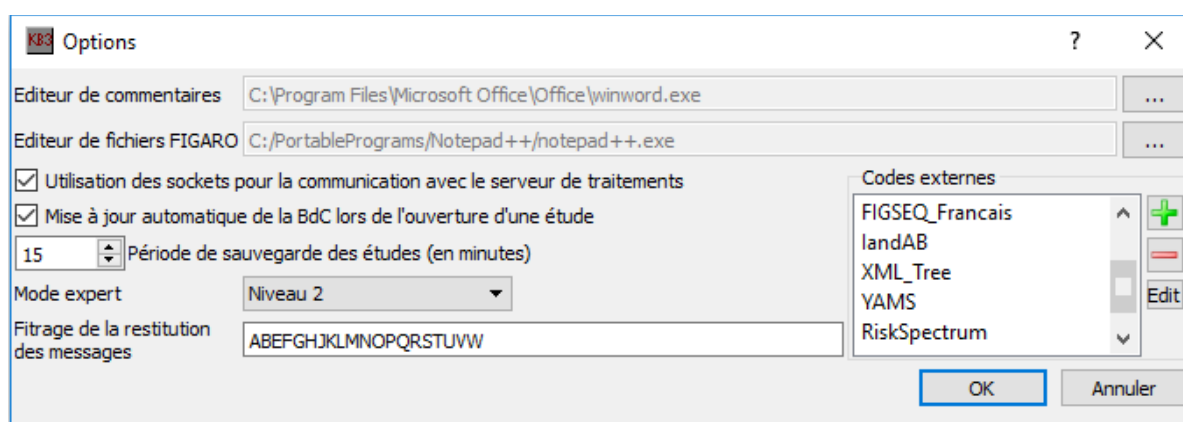
EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 10/22
--	--	------------------------------

Les types terminaux **liens** ont tous les mêmes caractéristiques : ils sont noirs, en trait plein d'épaisseur 1, terminés par une flèche et ils sont autorisés d'un port "out" à un port "in".

4.4.2 Spécification des traitements associés à la bdc

En ce qui concerne les traitements associés à la bdc, VisualFigaro génère par défaut des données (à rectifier éventuellement, surtout en ce qui concerne les groupes de règles) pour les trois types de traitements suivants : génération d'ADD, simulation interactive, instanciation en Figaro 0. Ces modèles de traitements (qui sont effectués par KB3 lui-même) peuvent être modifiés, supprimés s'ils n'ont pas de sens pour la bdc en construction, ou dupliqués et adaptés de façon à proposer à l'utilisateur plusieurs modèles "prêts à l'usage".

En outre, il est possible d'ajouter des lancements de codes externes qui permettent d'enchaîner automatiquement des opérations telles que la génération d'un ADD et/ou d'un modèle en Figaro 0 et le lancement d'un traitement externe (par exemple : RiskSpectrum ou YAMS). Pour configurer ou vérifier la liste des codes que l'on peut appeler depuis KB3, il faut utiliser le menu Outils/Panneau d'options (avec un niveau d'expert à 2) qui offre une IHM pour renseigner le fichier KB3Configuration.xml² (cette fonction n'existe que depuis 2018 : avant il fallait éditer le fichier directement, en dehors de KB3) :



La copie d'écran ci-dessus montre une liste de codes déclarés dans le fichier KB3Configuration.xml. Pour le détail de la configuration de lancement d'un code particulier, cf. l'annexe A.

Bien entendu, il faut aussi installer les codes de calcul qu'on a l'intention d'utiliser. Pour les trois codes de calcul Figseq, YAMS et GRIF.Tree, il existe des installateurs qui déclarent automatiquement ces codes dans le fichier KB3Configuration.xml³. Voici un exemple du fichier KB3Configuration.xml (les premiers champs du fichier servent à mémoriser les emplacements des dernières données utilisées par KB3) :

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<configuration_KB3 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
fichier_bdi="E:\KB3Data\db\DigDyn.mdb" fichier_bdci="E:\KB3Data\2 BDC\BDC
DigDyn\V1.0\1_Sources\English\DynDig.bdc"
xsi:noNamespaceSchemaLocation="KB3Configuration.xsd">
  <code>
    <nom>FIGSEQ</nom>
    <chemin_exe/>
    <chemin_IHM>C:\Program Files\EDF MRI Tools\FIGSEQ\figseq-gui.exe</chemin_IHM>
    <chemin_traducteur/>
  </code>
</configuration_KB3>
```

² Noter que VisualFigaro utilise aussi ce fichier pour savoir où se trouvent YAMS et Figseq.

³ En fait, il peut exister deux fichiers KB3Configuration.xml : un fichier situé dans C:\ProgramData\EDF MRI TOOLS, dont les données sont accessibles à tous les utilisateurs de la machine, et qui n'est pas modifié par KB3, et un fichier propre à l'utilisateur "nom_user", situé dans le répertoire : C:\Users\nom_user\AppData\Roaming\EDF MRI TOOLS. A chaque lancement, KB3 lit les deux fichiers et prend l'union des informations qu'il y trouve. En cas de conflit pour un code donné, il prend les informations du fichier propre à l'utilisateur.

Pour éviter des conflits, c'est le fichier propre à l'utilisateur qui est modifié par les installateurs des codes de calcul et par KB3. Ce fichier est réécrit par KB3 à chaque fin de session, sous la forme d'une ligne unique. Pour pouvoir le lire commodément, il faut le reformater avec un éditeur de XML.

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 11/22
--	--	------------------------------

```

<chemin_pretraitement/>
<chemin_posttraitement/>
<sous_repertoire>Figaro0</sous_repertoire>
<ADD_requis>FAUX</ADD_requis>
<Figaro0_requis>VRAI</Figaro0_requis>
<base_de_faits_requise>FAUX</base_de_faits_requise>
</code>
<code>
  <nom>YAMS</nom>
  <chemin_exe/>
  <chemin_IHM>c:\Program Files\EDF MRI Tools\YAMS\yams-gui.exe</chemin_IHM>
  <chemin_traducteur/>
  <chemin_pretraitement/>
  <chemin_posttraitement/>
  <sous_repertoire>Figaro0</sous_repertoire>
  <ADD_requis>FAUX</ADD_requis>
  <Figaro0_requis>VRAI</Figaro0_requis>
  <base_de_faits_requise>FAUX</base_de_faits_requise>
</code>
</configuration_KB3>

```

Si on prévoit de faire appel depuis la base de connaissances à des codes ne figurant pas dans KB3Configuration.xml, il faut compléter ce fichier directement ou à l'aide du panneau d'options de KB3.

Remarque : Dans la plupart des cas (et comme dans les exemples ci-dessus), le seul champ <chemin_xx> utile est <chemin_IHM>. Si d'autres champs doivent être remplis, il faut prendre garde au fait que pour le pré et le post traitement, il faut donner le nom du **répertoire** les contenant, car les noms des fichiers exécutables de ces programmes (à savoir "pretraitement.bat" et "posttraitement.bat") sont "en dur" dans KB3.

Le champ <sous_repertoire> donne le nom d'un sous-répertoire du répertoire KB3data/extern (KB3data désignant le répertoire qui contient la base de données chargée dans KB3) dans lequel seront créées les données temporaires nécessaires au lancement du code externe correspondant.

Les opérations préliminaires décrites précédemment étant faites, il reste maintenant à insérer dans le fichier .bdc de la base de connaissances en cours de construction (juste après le modèle de génération de Figaro 0) un (ou plusieurs) modèle(s) de lancement de code(s) externe(s).

Le seul lien entre la déclaration d'un traitement externe dans le fichier .bdc et le fichier KB3Configuration.xml est le nom du code. Il faut donc faire attention à ne pas se tromper en le saisissant dans l'IHM VisualFigaro (il n'y a pas de contrôle).

Voici un exemple de ce qui est introduit dans le fichier .bdc à la suite de la déclaration d'un traitement externe permettant de lancer Figseq sans avoir à faire manuellement la génération du Figaro 0 au préalable.

```

<MODELE_TRAITEMENT_EXTERNE>
  <NOM>Lancement FIGSEQ</NOM>
  <DESCRIPTION>Génère le figaro 0 et lance Figseq</DESCRIPTION>
  <CODES>
    <TOUS>FAUX</TOUS>
    <NOM_CODE>FIGSEQ</NOM_CODE>
  </CODES>
  <ADD>FAUX</ADD>
  <FIG0>VRAI</FIG0>
</MODELE_TRAITEMENT_EXTERNE>

```

On remarque qu'il y a une redondance d'information entre les champs

```

<ADD>FAUX</ADD>
<FIG0>VRAI</FIG0>

```

du fichier .bdc et les champs

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 12/22
--	--	------------------------------

```
<ADD_requis>FAUX</ADD_requis>
<Figaro0_requis>FAUX</Figaro0_requis>
```

du fichier KB3Configuration.xml.

Les informations du fichier .bdc sont prises en compte au moment de l’affichage de la fenêtre KB3 « Lancement traitement externe » pour griser certaines cases, mais quand on clique pour lancer effectivement le traitement, une vérification de conformité avec ce qui est mentionné dans le fichier KB3Configuration.xml est faite et déclenche éventuellement un message d’erreur. Il faut donc s’assurer que l’on respecte bien dans le fichier .bdc ce qui est spécifié pour le code à lancer dans le fichier KB3Configuration.xml.

Si des groupes de règles ont été définis dans le fichier .fi, on est amené à préciser dans le .bdc les groupes de règles à considérer pour chaque type de traitement. Il faut tenir compte du fait que les règles non attribuées à un (des) groupe(s) sont considérées comme faisant partie du groupe SANS_NOM. Si on ne précise pas quels groupes de règles il faut utiliser pour un traitement donné, *toutes* les règles sont utilisées. On peut déclarer les choix de groupes de règles faits dans la base de connaissance comme modifiables ou non par l'utilisateur de la base de connaissances dans KB3.

4.4.3 Premiers tests

On peut à ce stade charger la bdc dans KB3 et faire des tests en créant des objets. Tous les objets (nœuds et liens) peuvent être créés, et on peut brancher tout lien sur tout nœud, mais cela n'affecte pas les interfaces : il faut les remplir "à la main", à partir de l'éditeur d'objets. Les icônes sont toutes identiques à l'icône par défaut.

Après saisie de quelques objets, on peut lancer des traitements : il faut bien prendre garde aux choix des groupes de règles, pour chaque type de traitement. Suivant les résultats, on peut supprimer ou modifier les modèles de traitement.

Dans notre exemple, ce premier test nous permet de voir qu'il manque une condition dans la règle d'interaction qui propage l'attribut "relie" dans le type arete_uni_dir :

```
INTERACTION
SI relie DE depart ET MARCHE ET MARCHE DE arrivee
ALORS relie DE arrivee ;
```

4.5 Spécifier les points de connexion

L'IHM de KB3 permet de définir des interfaces graphiques « intelligentes » capables de connecter (ou refuser de connecter) un lien à un nœud en fonction des instructions contenues dans le fichier .bdc. Il est également possible de faire en sorte que le lien aille automatiquement s'accrocher à tel ou tel point de connexion (en remplissant une ou plusieurs interfaces d'objets FIGARO), sans obliger l'utilisateur à le viser précisément.

Afin que les instructions de remplissage d'interfaces aient un sens, il est nécessaire de mettre des contraintes sur les possibilités d'attacher un lien donné sur des points de connexion de nœuds donnés. Ces contraintes peuvent être déclarées au niveau des liens et/ou des nœuds, dans la sous-structure CONNEXION_ACCEPTEE.

Au niveau des nœuds, on définit des **points de connexion** (à ne pas confondre avec les interfaces FIGARO) nommés⁴, qui sont associés à une position graphique (coordonnées X et Y entre -100 et 100, qui sont des positions définies par rapport au centre (de coordonnées (0,0)) du rectangle inscrit autour de l'icône du nœud) et à la liste des liens dont l'accrochage sur le point de connexion sera autorisé.

De même, pour chaque lien, on peut préciser les points de connexion sur lesquels le lien pourra être accroché.

C'est ainsi que par exemple, au sein d'une même bdc, on pourra avoir un lien représentant un tuyau et allant d'un point de connexion « out » d'un type composant_hydraulique à un point de connexion « in » d'un type composant_hydraulique, et un autre lien représentant un câble électrique et allant d'un point de connexion « out » d'un type composant_electrique à un point de connexion « in » d'un type composant_electrique. Le même résultat serait obtenu en différenciant les noms des points de connexion suivant qu'ils appartiennent à des composants électriques ou hydrauliques, ce qui permettrait d'autoriser la connexion des liens sur le type FIGARO, qui est le père de tous les types. La première solution est préférable car elle évite l'inflation du vocabulaire de la bdc, qui est une source d'erreurs.

⁴ Les points de connexion sont appelés « ports » dans l'IHM VisualFigaro.

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 13/22
--	--	------------------------------

Deux philosophies différentes sont possibles pour placer géographiquement les points de connexion d'un nœud, suivant que l'on pense que l'utilisateur voudra viser des points précis ou non pour "accrocher" les liens de son modèle :

- on peut donner des contraintes discriminantes sur les types de liens qu'il est possible d'accrocher à chaque point de connexion, avec des règles de remplissage d'interfaces adaptées à chaque cas (dans ce cas, il sera inutile de viser, et il sera possible d'avoir plusieurs points de connexion superposés, par exemple au point (0, 0)),
- on peut aussi placer les points de connexion suffisamment éloignés les uns des autres pour qu'en visant un point précis l'utilisateur puisse décider de connecter un lien sur tel ou tel point de connexion, ce qui supprime la nécessité de déclarer des contraintes discriminantes. De toute façon, en cas d'ambiguïté (visée imprécise), un dialogue demandera à l'utilisateur en quel point il veut établir la connexion.

La première solution est évidemment la plus satisfaisante dans la majorité des cas ; elle a l'avantage de minimiser le nombre de noms de points de connexion à créer et donc à gérer par la suite. En particulier, elle permet de ne pas avoir à retoucher (sauf peut-être en ce qui concerne la position en X et Y) les points de connexion créés par défaut lors de la génération du squelette du fichier .bdc à partir du fichier .fi.

Cependant, il peut arriver qu'on ait besoin de pouvoir brancher des liens banalisés (par exemple des fils électriques) sur un quelconque des multiples points de connexion d'un composant (par exemple un circuit intégré avec de nombreuses pattes) ; dans ce cas, c'est la deuxième solution qu'il faut prendre.

4.6 Renseigner les contraintes de connexion et les règles de remplissage des interfaces

Les contraintes de connexion et les règles de remplissage des interfaces sont regroupées dans les structures <CONNEXION_ACCEPTEE>, qui sont des structures filles de <POINT_CONNEXION> pour les nœuds, et de <POINT_DEPART> ou <POINT_ARRIVEE> pour les liens.

Si c'est possible, ne renseigner les structures <CONNEXION_ACCEPTEE> que dans les liens. Cela permet de ne pas modifier les points de connexion standard "in" et "out" créés par défaut pour tous les nœuds.

Les autres avantages de cette façon de procéder sont les suivants :

- comme en général, il y a beaucoup moins de liens que de nœuds dans une base de connaissances, les informations <CONNEXION_ACCEPTEE>, qui sont les plus complexes d'un fichier .bdc, sont regroupées en un petit nombre de structures, ce qui facilite la vue d'ensemble.
- les règles de remplissage d'interface permettant de mettre l'objet dont un lien est issu dans une interface de l'objet sur lequel le lien arrive (ou réciproquement) ne peuvent être déclarées **que dans les liens**.

En fait, le principal avantage de l'utilisation des structures <CONNEXION_ACCEPTEE> dans des nœuds est de pouvoir factoriser des règles qui seraient répétées dans plusieurs liens différents. Mais cette situation paraît assez hypothétique.

Dans une structure <CONNEXION_ACCEPTEE>, **seul** le champ < TYPE_POINT_CONNEXION > est obligatoire. On peut spécifier le type FIGARO pour ne pas être restrictif.

Les règles de remplissage d'interface ont toutes des noms du type REGLE_XX_YY et elles sont associées à une ou plusieurs interfaces I1 à In.

Elles ont pour signification : remplir les interfaces I1 à In de XX par YY.

Exemple1 : l'extrait qui suit signifie que le point de départ du lien auquel appartient cet extrait peut être connecté sur le point de connexion "sortie" du type "nœud" (et ses sous-types).

Une telle connexion provoque le remplissage de l'interface "depart" du lien lui-même par le nœud de départ.

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 14/22
--	--	------------------------------

```

<POINT_DEPART>
  <CONNEXION_ACCEPTEE>
    <NOM>sortie</NOM>
    <TYPE_POINT_CONNEXION>noeud</TYPE_POINT_CONNEXION>
    <REGLE_LIEN_DEPART>
      <INTERFACE>depart</INTERFACE>
    </REGLE_LIEN_DEPART>
  </CONNEXION_ACCEPTEE>
</POINT_DEPART>

```

Exemple2 : l'extrait qui suit signifie que le point d'arrivée du lien auquel appartient cet extrait peut être connecté sur le point de connexion "entree" de n'importe quel type.

Une telle connexion provoque le remplissage de l'interface "arrivee" du lien lui-même par le nœud de départ.

```

<POINT_ARRIVEE>
  <CONNEXION_ACCEPTEE>
    <NOM>entree</NOM>
    <TYPE_POINT_CONNEXION>FIGARO</TYPE_POINT_CONNEXION>
    <REGLE_LIEN_ARRIVEE>
      <INTERFACE>arrivee</INTERFACE>
    </REGLE_LIEN_ARRIVEE>
  </CONNEXION_ACCEPTEE>
</POINT_ARRIVEE>

```

Exemple3 : Cet exemple plus complexe montre la possibilité de remplir plusieurs interfaces, à la fois dans les objets extrémités d'un lien et dans le lien lui-même. L'extrait qui suit signifie que le point de départ du lien auquel appartient cet extrait peut être connecté sur le point de connexion "sortie" du type "nœud".

Une telle connexion provoque le remplissage des interfaces "aretes" et "aretes_av" de l'objet de départ du lien (donc, un nœud) par le lien lui-même, et de l'interface "depart" du lien par le nœud de départ.

```

<POINT_DEPART>
  <CONNEXION_ACCEPTEE>
    <NOM>sortie</NOM>
    <TYPE_POINT_CONNEXION>noeud</TYPE_POINT_CONNEXION>
    <REGLE_DEPART_LIEN>
      <INTERFACE>aretes</INTERFACE>
      <INTERFACE>aretes_av</INTERFACE>
    </REGLE_DEPART_LIEN>
    <REGLE_LIEN_DEPART>
      <INTERFACE>depart</INTERFACE>
    </REGLE_LIEN_DEPART>
  </CONNEXION_ACCEPTEE>
</POINT_DEPART>

```

Remarque : quand une structure <CONNEXION_ACCEPTEE> contient plusieurs règles de remplissage d'interfaces, ces règles doivent être dans l'ordre suivant d'après le schéma définissant la structure du fichier .bdc, faute de quoi la bdc ne pourra pas être chargée dans KB3.

1. REGLE_NOEUD_LIEN
2. REGLE_LIEN_NOEUD
3. REGLE_DEPART_LIEN
4. REGLE_LIEN_DEPART
5. REGLE_ARRIVEE_LIEN

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 15/22
--	--	------------------------------

6. REGLE_LIEN_ARRIVEE
7. REGLE_ARRIVEE_DEPART
8. REGLE_DEPART_ARRIVEE

Exemple4 : cet exemple est pris dans une base de connaissances contenant un type composant, muni d'un point de connexion "sortie" et d'un point de connexion "entree". Tous les sous-types de composant héritent de ces points de connexion, à l'exception du composant "bascule", qui hérite du point "sortie", mais a, au lieu du point de connexion "entree", deux points appelés "entree_s" et "entree_n". On a besoin dans cette base de connaissances d'un seul type de lien, qui va remplir, **en fonction du point de connexion de destination**, soit l'interface "normal", soit l'interface "secours", et dans tous les cas l'interface "amont" du nœud auquel il arrive.

```

<LIEN>
  <POINT_DEPART>
    <CONNEXION_ACCEPTEE>
      <NOM>sortie</NOM>
      <TYPE_POINT_CONNEXION>composant</TYPE_POINT_CONNEXION>
    </CONNEXION_ACCEPTEE>
  </POINT_DEPART>
  <POINT_ARRIVEE>
    <CONNEXION_ACCEPTEE>
      <NOM>entree</NOM>
      <TYPE_POINT_CONNEXION>composant</TYPE_POINT_CONNEXION>
      <REGLE_ARRIVEE_DEPART>
        <INTERFACE>amont</INTERFACE>
      </REGLE_ARRIVEE_DEPART>
    </CONNEXION_ACCEPTEE>
    <CONNEXION_ACCEPTEE>
      <NOM>entree_n</NOM>
      <TYPE_POINT_CONNEXION>bascule</TYPE_POINT_CONNEXION>
      <REGLE_ARRIVEE_DEPART>
        <INTERFACE>normal</INTERFACE>
        <INTERFACE>amont</INTERFACE>
      </REGLE_ARRIVEE_DEPART>
    </CONNEXION_ACCEPTEE>
    <CONNEXION_ACCEPTEE>
      <NOM>entree_s</NOM>
      <TYPE_POINT_CONNEXION>bascule</TYPE_POINT_CONNEXION>
      <REGLE_ARRIVEE_DEPART>
        <INTERFACE>secours</INTERFACE>
        <INTERFACE>amont</INTERFACE>
      </REGLE_ARRIVEE_DEPART>
    </CONNEXION_ACCEPTEE>
  </POINT_ARRIVEE>
</LIEN>

```

4.7 Spécifier les algorithmes d'objets déduits

Une fois qu'il est facile de construire des modèles de systèmes, il faut, pour éviter de devoir les construire à la main, spécifier les algorithmes qui vont construire automatiquement les objets déduits (nouveaux objets dont les caractéristiques, interfaces, etc. sont remplis à partir des objets saisis par l'utilisateur).

Ces algorithmes sont définis en un langage qui est une extension du langage FIGARO, autorisant des actions spéciales de type création ou modification des champs d'objets FIGARO. Ce langage fait l'objet d'une documentation spécifique.

Remarque : les objets déduits ne sont pas une nécessité. Seules les bases de connaissances complexes y font appel.

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 16/22
----------------------------	--	------------------------------

4.8 Choisir ou créer des icônes et styles de liens

Une fois parvenu à ce stade, le .bdc est suffisant pour permettre de saisir rapidement des systèmes et pour lancer différents types de traitement. Les objets créés ont tous le même aspect, mais ils portent des noms formés du nom de leur type suivi d'un numéro, ce qui est suffisant pour les distinguer. Toutefois, il devient rapidement utile pour la lisibilité des modèles, de particulariser les icônes et styles de liens associés aux différents types.

4.8.1 Icônes

Se servir de l'IHM pour choisir des icônes standard (mises dans le répertoire icons lors de la création de la base de connaissances) ou aller en récupérer d'autres (qui dans ce cas vont être copiées dans le répertoire icons) dans d'autres répertoires.

Remarque : outre les icônes correspondant aux objets définis dans la bdc, il faut systématiquement avoir une icône de nom MCG (donc les deux fichiers MCG.ico et MCG.sym) qui servira à représenter les macro-composants graphiques.

4.8.2 Liens

Se servir de l'IHM pour choisir des icônes standard (mises dans le répertoire icons lors de la création de la base de connaissances) ou aller en récupérer d'autres (qui dans ce cas vont être copiées dans le répertoire icons) dans d'autres répertoires, et éditer les caractéristiques du lien accessibles par le bouton "Edit Default GV".

Note importante : il faut recharger la base de connaissances dans KB3 pour prendre en compte des changements d'icônes. En revanche, il n'est pas nécessaire de ressaisir les systèmes.

4.9 Exploiter l'héritage dans le fichier .bdc

Jusqu'ici, nous n'avons exploité l'héritage que dans le fichier en langage Figaro. Il est possible de le faire aussi dans le .bdc. Dans ce dernier, les liens d'héritage doivent être explicites, et doivent constituer **un sous-ensemble des liens d'héritage existants dans le fichier .fi**. Il est possible de sauter des étapes dans la hiérarchie, car il n'est pas obligatoire de déclarer les types non terminaux dans le .bdc.

Par exemple, si la hiérarchie des types est la suivante dans le fichier .fi :

```
-FIGARO
| -etat
|   | -etat_depart
|   |   | -etat_depart_tangible
|   |   | -etat_depart_instantane
|   | -etat_panne
| -transition_exp
|   | -defaillance
|   | -reparation
```

On pourra spécifier que `etat_depart_tangible` et `etat_depart_instantane` héritent certaines de leurs caractéristiques (variantes graphiques ou points de connexion) de `etat`.

Il n'existe pas de notion d'héritage permettant de "factoriser" des points de connexion pour les liens ; en revanche, on peut factoriser les variantes graphiques.

Il est indispensable d'utiliser l'héritage pour éviter des répétitions qui à terme sont génératrices d'erreurs car la non utilisation de l'héritage fait énormément gonfler le volume d'informations à gérer.

Il semble que l'introduction de l'héritage est optimale à ce point de développement de la bdc. Lorsqu'elle est encore très instable car en cours de mise au point, il semble difficile de le faire, car des changements d'héritage dans le fichier .fi nécessiteraient à chaque fois des modifications dans le .bdc.

Note importante : depuis 2018, il est possible d'utiliser l'héritage de manière classique (donc comme dans le fichier .fi) dans le .bdc. Cette nouvelle possibilité permet de réduire, souvent par un facteur 2 ou 3, la taille du fichier .bdc. Il suffit de déclarer l'attribut AUTOH et de le mettre à VRAI dans le premier tag:

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 17/22
--	--	------------------------------

```
<BDC xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="bdcfr.xsd" AUTOH="VRAI">
```

pour utiliser ce nouveau mode de gestion de l'héritage, et que le fichier .bdc soit correctement interprété par KB3.

Cette option n'est pas disponible dans VisualFigaro. Elle est seulement accessible avec le nouvel éditeur FigaroIDE de la société Aristè, qui a l'avantage d'automatiser le passage du mode AUTOH = "FAUX" à AUTOH = "VRAI".

4.10 Ajouter des modes de visualisation et variantes graphiques

Ce point est celui qui va permettre d'avoir une application vraiment conviviale, voire spectaculaire, avec changements d'aspect du modèle graphique au cours d'une simulation interactive.

Il faut commencer par déclarer la liste des noms de tous les modes de visualisation que l'on prévoit.

L'IHM permet ensuite de choisir parmi ces modes ceux dans lesquels une variante graphique s'applique.

Une variante graphique permet de :

- Changer la couleur d'une icône
- Faire clignoter l'icône
- Changer d'icône (il faut alors disposer d'un autre fichier .sym que l'on associe à la variante graphique)
- Inscrire un texte à proximité de l'icône (position HAUT, BAS, GAUCHE ou DROITE). Cela permet par exemple d'afficher la valeur d'une variable en la désignant par %s au sein d'une chaîne de caractères définissant un "format". Voici par exemple comment afficher dans tous les cas (condition égale à VRAI) un texte comportant des variables et des chaînes de caractères fixes concaténées, pour afficher une première variable, numérique et une deuxième de type énuméré qui peut prendre les valeurs 'watts' et 'megawatts' :

```
<VARIANTE_GRAPHIQUE>
  <NOM>VG2</NOM>
  <VISUALISATION>Capacités</VISUALISATION>
  <CONDITION>VRAI</CONDITION>
  <TEXTE>
    <EXPRESSION>
      <FORMAT>Puissance = %s %s</FORMAT>
      <PARAMETRES>capacite</PARAMETRES>
      <PARAMETRES>unite</PARAMETRES>
    </EXPRESSION>
    <POSITION_TEXTE>HAUT</POSITION_TEXTE>
  </TEXTE>
</VARIANTE_GRAPHIQUE>
```

Cela écrira par exemple : Puissance = 15,5 megawatts

Chaque variante graphique est conditionnée à la fois par le mode de visualisation choisi par l'utilisateur de KB3 et par une condition booléenne définie en langage Figaro. Si plusieurs variantes graphiques sont applicables dans une situation donnée, seule la dernière est appliquée (les effets des variantes ne sont pas cumulatifs).

Il faut utiliser au maximum l'héritage pour les variantes graphiques.

4.11 Ajouter des calques⁵

Afin de pouvoir basculer rapidement entre différents affichages, plus ou moins épurés, d'un schéma complexe, on peut répartir les objets dans différents calques, que l'on choisit d'afficher ou non. On peut aussi gérer l'ordre des calques, pour tenir compte de situations où il y a des superpositions d'éléments de dessin.

Chaque calque peut contenir une image de fond (un fichier de format .bmp, à l'exclusion de tout autre format) et une liste d'objets. Certains objets peuvent appartenir à plusieurs calques. L'utilisateur de KB3 peut définir dans chaque étude ses propres calques, mais pour lui faciliter le travail on peut déclarer dans la base de connaissances

⁵ A partir de KB3 v1.3.2.4.

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 18/22
--	--	------------------------------

une liste de calques, avec les types (et ceux qui en héritent) à afficher sur ces calques. Ainsi, lors de la création dans KB3 d'un nouvel objet, celui-ci sera automatiquement associé à certains calques, en fonction des informations définies dans la bdc. Les types déclarés dans les calques doivent être présents par ailleurs dans le fichier .bdc.

Voici un exemple de définition de deux calques, l'un contenant une image de fond ainsi que les composants électriques et hydrauliques, et l'autre contenant le contrôle-commande d'un système (les calques sont à déclarer juste après la liste des modes de visualisation dans le .bdc) :

```
<CALQUE>
  <NOM>CP</NOM>
  <DESCRIPTION>Composants principaux</DESCRIPTION>
  <FICHIER>image_de_fond.bmp</FICHIER>
  <TYPE>cpt_electrique</TYPE>
  <TYPE>cpt_hydraulique</TYPE>
</CALQUE>
<CALQUE>
  <NOM>CC</NOM>
  <DESCRIPTION>Contrôle-commande</DESCRIPTION>
  <TYPE>controle_commande</TYPE>
</CALQUE>
```

4.12 Verrouiller l'application

Il est temps de revenir sur le fichier .fi en ajoutant des facettes EDITION permettant de :

- cacher à l'utilisateur final les champs dont il n'aura pas besoin (EDITION NON VISIBLE),
- empêcher toute modification de champ qui risquerait d'introduire des incohérences dans un modèle saisi (EDITION NON MODIFIABLE). C'est en particulier le cas des interfaces qui sont remplies automatiquement par le tracé de liens. Il est **très important** de déclarer ce genre d'interface comme NON MODIFIABLE, car outre le fait que cela évitera des erreurs dues à de mauvaises manipulations par l'utilisateur, cela aura pour effet de remplir correctement les interfaces des objets collés à la suite d'un copier-coller d'un groupe d'objets contenant des liens.

Eventuellement, il est aussi souhaitable d'ajouter des facettes CARDINAL pour les interfaces pour lesquelles ce n'est pas encore fait, de façon à contrôler le nombre d'objets qui y seront déclarés.

4.13 Rédiger les aides à l'utilisateur

Deux types d'aide sont souhaitables : une aide hors ligne et une aide en ligne.

Il est de toute façon souhaitable de proposer à l'utilisateur un document indépendant de KB3, mais présenté de manière que l'on puisse rapidement y retrouver les définitions des types, avec leurs champs, etc. Un fichier contenant un squelette de document d'aide est automatiquement copié dans le répertoire de la base de connaissances lors de sa création. Une autre possibilité est de créer un fichier Excel dans lequel chaque feuille est associée à un type de la bdc, et une feuille « sommaire » contient la hiérarchie des types.

Pour l'aspect proprement « en ligne », il faut renseigner le champ DESCRIPTION associé à chaque type dans le fichier .bdc, car ce champ s'affiche à droite de l'icône du type dans la palette. De même, il est possible de renseigner dans le fichier .fi les champs LIBELLE associés aux différentes caractéristiques des types : interfaces, constantes, attributs, effets, pannes. En effet, ces libellés, outre le fait qu'ils servent à donner des noms lisibles aux portes des arbres de défaillances générés par KB3, sont affichés par l'IHM de KB3 lorsqu'on clique sur le "+" associé à un nom d'objet dans la palette. Ces champs LIBELLE bénéficient de l'héritage au même titre que les constantes ou variables qu'ils décrivent.

5 Modification d'une bdc existante

5.1 Principes généraux

Le risque principal lorsqu'on reprend une base existante, est de se perdre dans les détails et d'introduire des erreurs par manque de vue d'ensemble. C'est pourquoi nous conseillons, pour une reprise assez profonde d'une bdc, de commencer par supprimer le fichier .bdc de façon à re-générer un squelette de fichier .bdc qui sera réduit au minimum, et de reprendre les étapes définies dans cette note, sans en sauter aucune. Il n'est pas interdit,

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 19/22
--	--	------------------------------

évidemment, de gagner du temps à chaque étape en utilisant le copier-coller à partir de l'ancienne version du fichier .bdc (cela se fait alors en utilisant QXmlEdit).

Remarque : gestion des versions d'une bdc. Dans le cas où on fait peu d'évolutions, il est pratique de garder le même nom de bdc (ce nom est déclaré dans le fichier .bdc), et de signaler le numéro de version dans la description. Le nom de la bdc et sa description sont affichés dans KB3; on ne peut donc pas se tromper sur la version de bdc que l'on utilise. En revanche, pour des évolutions significatives, il vaut mieux inclure le numéro de version dans le nom de la bdc. Dans tous les cas, il est recommandé de conserver les mêmes noms pour les fichiers.

5.2 Comment recharger les études saisies avec une version précédente de bdc ?

Même si on modifie assez fortement une base de connaissances, on aimerait pouvoir réutiliser les études saisies avec une version précédente plutôt que de les ressaisir intégralement.

On peut procéder ainsi pour gérer les changements les plus courants :

- changements de règles et de variables déduites uniquement : on doit pouvoir recharger une étude directement ;
- changements de noms de variables essentielles : les initialisations de ces variables faites dans les profils et variantes de KB3 seront perdues au chargement de l'étude ;
- changements de noms d'interfaces : les remplissages de ces interfaces seront perdus au chargement de l'étude. Cela peut introduire des incohérences relativement difficiles à détecter dans une grosse étude ;
- suppression pure et simple de types : les objets des types inexistants dans la nouvelle bdc seront détruits au chargement de l'étude ;
- changements de noms de types visibles dans la palette. Pour renommer le type A en B, on peut ajouter à l'ancienne base de connaissances le type B avec toute sa définition, charger l'étude avec cette base de connaissances provisoire et utiliser KB3 pour changer le type de tous les objets du type A en objets du type B (c'est possible en une seule opération). On perd éventuellement des informations lors de cette transformation. On sauve ensuite l'étude, puis on la rouvre avec la nouvelle base de connaissances.

Dans tous les cas, KB3 liste dans son fichier log toutes les informations détruites lors du chargement de l'étude avec une nouvelle base de connaissances. On peut donc par un examen détaillé de ce fichier être informé de tous les changements qu'il faut faire. Pour des changements très nombreux et répétitifs, cela peut valoir la peine d'écrire un petit script en SQL à charger au lancement de KB3 (par le bouton "appliquer patch").

Voici ci-dessous un exemple de tel script, dont l'objet est de réunir en un seul type "lien_testeur" des objets qui appartenaient à deux types différents (lien_testeur_hydrau et lien_testeur_elec). Il s'agit donc juste de changer le nom de type des anciens liens. Un tel script doit être dans un fichier d'extension .kbp.

```
<?xml version="1.0" encoding="utf-8"?>
<DatabaseManagement xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="KB3DatabaseManager.xsd">
  <VersionUpdate fromVersion="24" toVersion="24">
    <Comment>modification noms bdc et types</Comment>

    <Command>update STRU_FIG set NOM="lien_testeur" where
      NOM="lien_testeur_hydrau";</Command>
    <Command>update STRU_FIG set TYPE_NAME="lien_testeur" where
      TYPE_NAME="lien_testeur_hydrau";</Command>

    <Command>update STRU_FIG set NOM="lien_testeur" where
      NOM="lien_testeur_elec";</Command>
    <Command>update STRU_FIG set TYPE_NAME="lien_testeur" where
      TYPE_NAME="lien_testeur_elec";</Command>

  </VersionUpdate>
</DatabaseManagement>
```

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 20/22
--	--	------------------------------

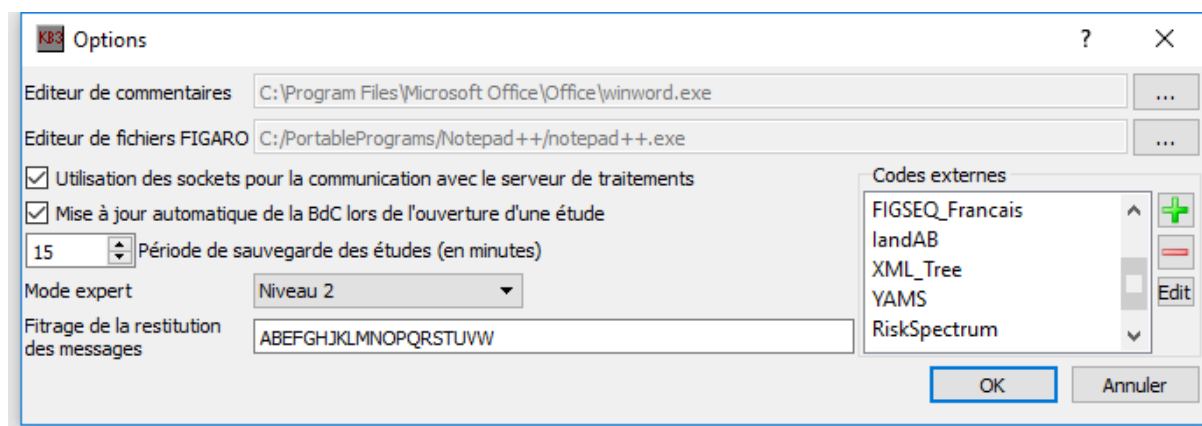
6 Conclusion

Les conseils que nous avons donnés dans ce document devront être affinés et complétés au fur et à mesure de :

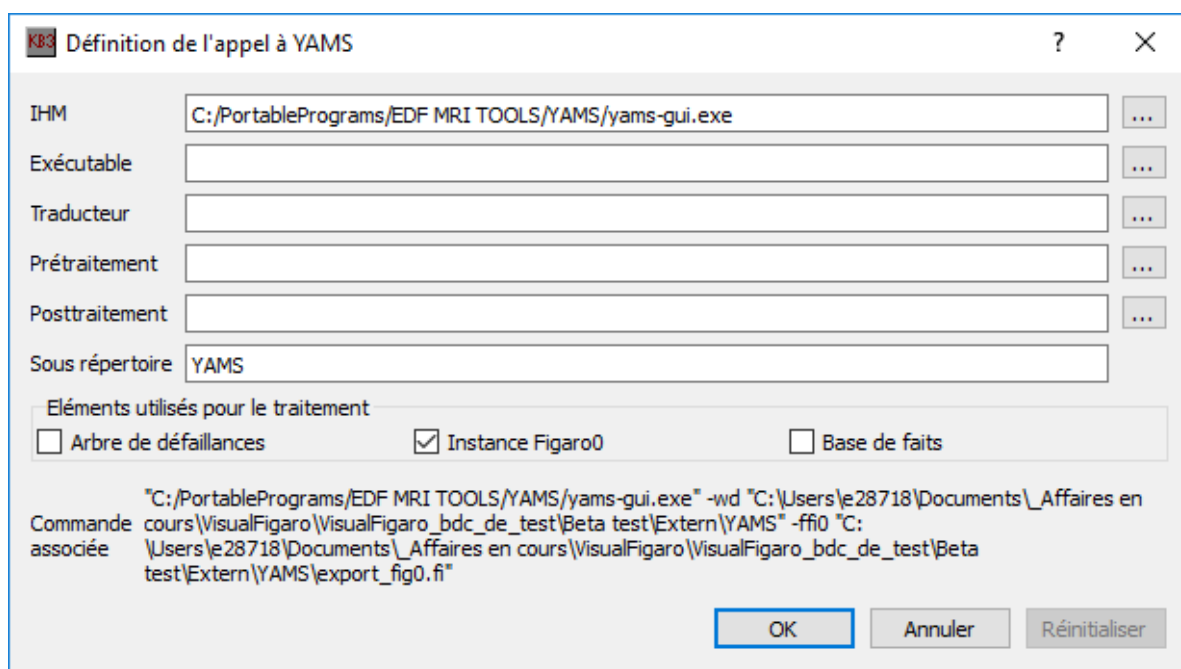
- l'évolution de KB3 et de l'éditeur VisualFigaro,
- l'accumulation de nouvelles expériences,
- l'apparition de nouveaux outils sur le marché.

7 Annexe A : passage de paramètres lors du lancement d'un code externe par KB3

KB3 passe des paramètres aux codes externes qu'il lance. On peut préciser ces paramètres dans le panneau d'options. Il faut d'abord sélectionner un code dans la fenêtre suivante (ou le créer en cliquant sur le bouton "+" s'il n'est pas dans la liste) :



...puis cliquer sur le bouton "Edit" pour accéder au panneau suivant (ici pour YAMS) :



Cela permet dans cet exemple de lancer YAMS en lui faisant charger directement le fichier Figaro 0 qui vient d'être généré par KB3 (on peut faire la même chose pour Figseq).

En fonction des informations choisies dans le panneau, KB3 affiche en bas la commande qu'il lancera lorsque l'utilisateur choisira le traitement associé à ce nom de code (ici, YAMS). Cela permet de savoir comment le programme qui sera lancé devra interpréter les arguments qui lui seront passés.

Si le code externe lancé est un fichier .bat, ce fichier peut accéder aux paramètres par %1, %2 etc. Dans l'exemple ci-dessus, %1 vaut "-wd", %2 est le répertoire de travail, %3 vaut "-ffi0" et %4 est le nom complet d'accès au fichier Figaro0 à charger par YAMS.

EDF R&D Département MRI	Manuel de développement de bases de connaissances pour le logiciel KB3 v3	HT-52/03/042/J Page 22/22
--	--	------------------------------

Voici un exemple : le lancement du traducteur ADD2BDMP qui transforme un arbre de défaillances généré par KB3 (avec certaines particularités permettant de coder la présence de gâchettes) en un BDMP écrit en Figaro 0.

C'est un fichier .bat d'une ligne contenant la commande suivante :

```
"C:\EDF MRI TOOLS\add2bdmp\add2bdmp.exe" "C:\EDF MRI TOOLS\add2bdmp\BDMP_Fr.fi" "C:\EDF MRI TOOLS\add2bdmp\OpaleFr.xml" %4 -o %2\BDMPfig0.fi
```

Ce fichier .bat lance le programme add2bdmp.exe avec en arguments (dans l'ordre) :

- Un fichier contenant la base de connaissances BDMP_Fr.fi
- Le fichier OpaleFr.xml de paramétrage indiquant comment les gâchettes du BDMP sont codées dans les noms de portes et feuilles de l'arbre de défaillances
- Le 4^{ème} argument envoyé par KB3 : c'est le nom complet (avec son chemin) du fichier d'arbre de défaillances généré par KB3
- Le 2^{ème} argument envoyé par KB3 (nom du répertoire contenant le fichier d'arbre de défaillances généré) concaténé avec "BDMPfig0.fi" afin de former le nom du fichier de sortie du programme add2bdmp.exe.